

UNIVERSITÉ HASSAN 1ER

Faculté des Sciences et Techniques, Settat

PROGRAMMATION SYSTÈME ET RÉSEAU

---

## Rapport de Travaux Pratiques 3

Signaux et Communication Inter-Processus (Tubes)

---

PRÉSENTÉ PAR :

EZZAHHANY YASSINE

FIGI - Semestre 4 (S4)

## Table des Matières

---

1. Introduction Générale
2. Exercice 1 : Capture de SIGINT et Sauvegarde de Données
3. Exercice 2 : Écriture dans un tube sans lecteur (Analyse de SIGPIPE)
4. Exercice 3 : Communication Bidirectionnelle Père/Fils via Tubes
5. Conclusion Générale

# 1. Introduction Générale

---

Ce travail pratique est dédié à l'étude des mécanismes de communication et d'interaction de bas niveau fournis par les noyaux système de type Unix (POSIX). La programmation système ne se limite pas à la simple création concourante de processus ; elle requiert également la mise en œuvre de canaux d'échange de données sécurisés (les tubes) et la gestion d'événements asynchrones transmis par le système (les signaux).

À travers les trois exercices proposés, nous abordons la modification du comportement par défaut d'un processus lors de l'envoi du signal d'interruption `SIGINT`, l'étude d'une anomalie classique de rupture de flux menant à l'envoi de `SIGPIPE`, et enfin la mise en place d'une topologie de communication bidirectionnelle entre deux processus parent/fils à l'aide de primitives systèmes `pipe`.

## 2. Exercice 1 : Capture de SIGINT et Sauvegarde de Données

---

### Analyse & Objectifs

L'exercice demande de modifier le comportement standard du processus lors de la réception du signal `SIGINT` (couramment envoyé via `Ctrl-C` dans le terminal). Au lieu de s'arrêter brusquement, le processus doit intercepter le signal, effectuer une écriture d'urgence (sauvegarde de la progression courante) dans le fichier `sauvegarde.txt`, et se terminer proprement.

Pour réaliser cette tâche en respectant les spécifications de sécurité POSIX les plus strictes :

- Nous utilisons la structure et la fonction `sigaction()` qui s'avèrent fiables et évitent les effets de bord indésirables ou les pertes de gestionnaires de l'ancien appel `signal()`.
- Nous veillons à ce que le gestionnaire de signal (qui s'exécute de façon asynchrone) n'appelle que des fonctions garanties **Async-Signal-Safe** (réentrantes). Les fonctions de bibliothèque de haut niveau comme `fopen()`, `fprintf()` ou `printf()` ne le sont pas car elles manipulent des tampons globaux non protégés. Nous utilisons donc à leur place les appels système directs `open()`, `write()` et `close()`.

**Code Source** ( `exercice1.c` )

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <fcntl.h>
#include <string.h>

// Variable globale volatile pour garantir l'accès direct et atomique lors du signal
volatile sig_atomic_t iteration_count = 0;

// Conversion manuelle d'un entier en chaîne (conforme "async-signal-safe")
void int_to_str(int n, char *str) {
    int i = 0;
    if (n == 0) {
        str[i++] = '0';
    } else {
        while (n > 0) {
            str[i++] = (n % 10) + '0';
            n /= 10;
        }
    }
    str[i] = '\0';
    for (int j = 0; j < i / 2; j++) {
        char temp = str[j];
        str[j] = str[i - 1 - j];
        str[i - 1 - j] = temp;
    }
}

// Gestionnaire du signal SIGINT
void handle_sigint(int sig) {
    (void)sig; // Éviter l'avertissement de compilation

    const char *filename = "sauvegarde.txt";

    // O_WRONLY | O_CREAT | O_TRUNC : Remplace/crée le fichier de sauvegarde
    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        const char *err = "Erreur de création de sauvegarde.txt\n";
        write(STDERR_FILENO, err, strlen(err));
        exit(EXIT_FAILURE);
    }
}

```

```

}

const char *header = "=== SAUVEGARDE ETAT PROGRAMME ===\n";
const char *status = "Statut : Arrêté via SIGINT (Ctrl-C)\n";
const char *iter_msg = "Derniere iteration atteinte : ";

write(fd, header, strlen(header));
write(fd, status, strlen(status));
write(fd, iter_msg, strlen(iter_msg));

char count_buf[12];
int_to_str(iteration_count, count_buf);
write(fd, count_buf, strlen(count_buf));
write(fd, "\n=====\\n", 35);

close(fd);

const char *console = "\\n[Signal] SIGINT intercepte. Sauvegarde dans 'sauvegarde";
write(STDOUT_FILENO, console, strlen(console));

exit(EXIT_SUCCESS);
}

int main() {
printf("=====\\n");
printf(" Exercice 1 : Capture de SIGINT et Sauvegarde de données\\n");
printf("=====\\n");
printf("[PID : %d] Programme en cours. Appuyez sur Ctrl-C...\\n\\n", getpid());

// Liaison du signal SIGINT à notre gestionnaire propre via sigaction
struct sigaction sa;
sa.sa_handler = handle_sigint;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;

if (sigaction(SIGINT, &sa, NULL) == -1) {
perror("Erreur sigaction");
return EXIT_FAILURE;
}

// Tâche fictive infinie incrémentant un compteur de progression
while (1) {

```

```
        iteration_count++;  
        printf("  -> Etape %d en cours...\n", iteration_count);  
        sleep(1);  
    }  
  
    return EXIT_SUCCESS;  
}
```

## Ressenti et Fonctionnement Pratique

Lorsqu'on lance l'exécution, le terminal affiche la progression séquentielle. Dès que l'utilisateur appuie sur `Ctrl-C`, le flux d'exécution principal s'arrête, le handler est invoqué, le fichier est écrit, et le programme s'achève de façon ordonnée. Une lecture ultérieure de `sauvegarde.txt` confirme la réussite de l'écriture.

### 3. Exercice 2 : Écriture dans un tube sans lecteur (Analyse de SIGPIPE)

---

#### Analyse du Déroulement du Code

Le code proposé effectue les actions successives suivantes :

Il réserve un espace pour un tube `tube[2]` de deux descripteurs.

Il instancie un tube via `pipe(tube)` .

Il ferme explicitement l'extrémité de lecture du tube : `close(tube[0])` . Désormais, le tube n'a plus **aucun lecteur actif**.

Il essaie de copier la chaîne `"AZERTYUIOP"` sur le descripteur d'écriture restant : `write(tube[1], buffer, ...)` .

#### Analyse Système et Réponse

Dans les spécifications système d'Unix, écrire dans un tube ne disposant plus de lecteurs actifs est considéré comme une anomalie grave : les octets écrits ne pourront jamais être consommés.

Pour signaler ce dysfonctionnement à l'application sans gaspiller de mémoire système ou bloquer le processus, le noyau Unix envoie immédiatement le signal `SIGPIPE` (**Signal 13 - Broken Pipe**) au processus écrivain.

Par conséquent, la réponse à la question posée est :

**En cas d'erreur (lors du write dans le tube sans lecteur), le code fait l'objet du signal `SIGPIPE` (Signal de numéro 13).**

Puisque ce signal n'est pas intercepté par le programme (aucun gestionnaire n'est installé), la sentence par défaut est appliquée par l'OS : le programme est **tué sur le coup**. Ainsi, la fonction d'affichage final `fprintf(stdout, "Fin du programme \n")` n'est **jamais atteinte**.

## 4. Exercice 3 : Communication Bidirectionnelle Père/Fils via Tubes

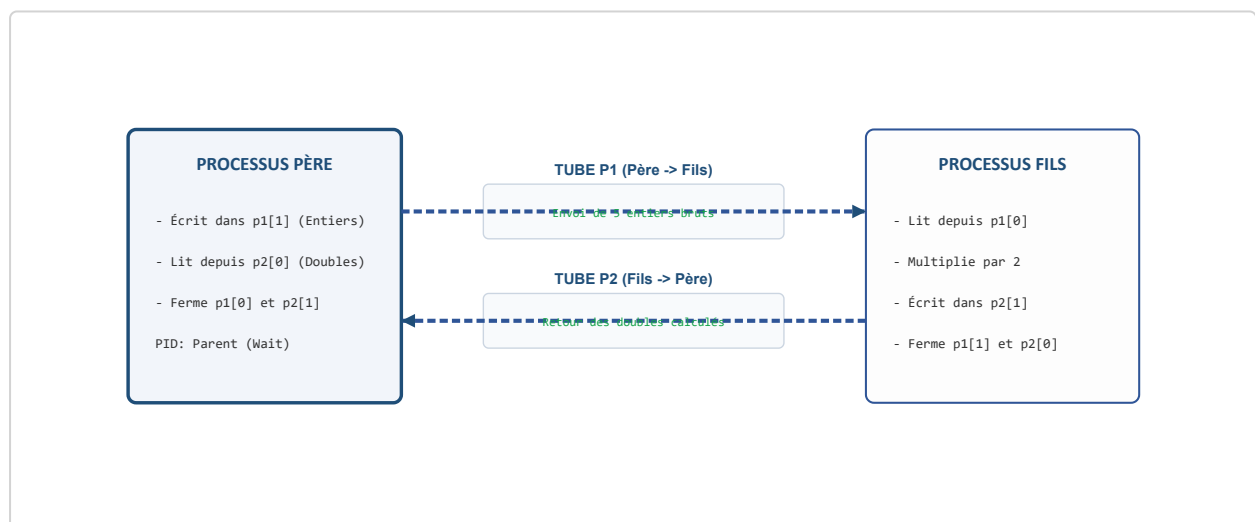
### Analyse & Topologie de Communication

Un tube (pipe) Unix est par nature **unidirectionnel**. L'échange d'informations dans les deux sens (Père vers Fils ET Fils vers Père) nécessite par conséquent l'utilisation de **\*\*deux tubes distincts\*\*** :

- **Le tube `p1` (Père vers Fils)** : Le père écrit ses entiers sur `p1[1]` et le fils les lit depuis `p1[0]`.
- **Le tube `p2` (Fils vers Père)** : Le fils écrit les valeurs doublées sur `p2[1]` et le père les lit depuis `p2[0]`.

Pour éviter tout blocage ou interférence, il est impératif que chaque processus ferme systématiquement les descripteurs de fichiers dont il n'a pas l'usage.

### Schéma de l'Architecture Réseau/IPC



Code Source ( `exercice3.c` )

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    int p1[2]; // Tube Père -> Fils
    int p2[2]; // Tube Fils -> Père

    printf("=====\\
    printf("  Exercice 3 : Communication bidirectionnelle via tubes (Père <-> Fils)
    printf("=====\\

    // Création des deux tubes
    if (pipe(p1) != 0 || pipe(p2) != 0) {
        perror("Erreur lors de l'initialisation des tubes");
        return EXIT_FAILURE;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("Erreur fork");
        return EXIT_FAILURE;
    }
    else if (pid == 0) {
        // --- PROCESSUS FILS ---
        close(p1[1]); // Ferme l'écriture sur p1 (Le fils ne fait que lire p1)
        close(p2[0]); // Ferme la lecture sur p2 (Le fils ne fait qu'écrire p2)

        printf("[Fils - PID: %d] Prêt à lire sur p1 et écrire sur p2.\\n", getpid());

        int val_recue;
        int val_double;

        for (int i = 0; i < 5; i++) {
            // Lecture bloquante de l'entier envoyé par le père
            if (read(p1[0], &val_recue, sizeof(int)) <= 0) {
                perror("[Fils] Erreur lecture");
                exit(EXIT_FAILURE);
            }
        }
    }
}

```

```

    }
    printf("[Fils - PID: %d] Reçu : %d\n", getpid(), val_recue);

    // Calcul imposé par l'énoncé
    val_double = val_recue * 2;

    // Renvoi immédiat sur p2
    if (write(p2[1], &val_double, sizeof(int)) <= 0) {
        perror("[Fils] Erreur écriture");
        exit(EXIT_FAILURE);
    }
}

close(p1[0]);
close(p2[1]);
printf("[Fils - PID: %d] Travail terminé.\n", getpid());
exit(EXIT_SUCCESS);
}
else {
    // --- PROCESSUS PÈRE ---
    close(p1[0]); // Ferme la lecture sur p1 (Le père ne fait qu'écrire p1)
    close(p2[1]); // Ferme l'écriture sur p2 (Le père ne fait que lire p2)

    int valeurs_test[5] = {5, 12, 23, 42, 99};
    int resultats[5];

    printf("[Père - PID: %d] Début de la transmission bidirectionnelle...\n\n",

    for (int i = 0; i < 5; i++) {
        printf("[Père] Envoi : %d\n", valeurs_test[i]);

        // Écriture de l'entier
        if (write(p1[1], &valeurs_test[i], sizeof(int)) <= 0) {
            perror("[Père] Erreur écriture");
            return EXIT_FAILURE;
        }

        // Attente bloquante du retour
        if (read(p2[0], &resultats[i], sizeof(int)) <= 0) {
            perror("[Père] Erreur lecture");
            return EXIT_FAILURE;
        }
    }
}

```

```
        printf("[Père] Reçu en retour : %d\n\n", resultats[i]);
    }

    // Synchronisation finale pour éviter les processus zombies
    wait(NULL);

    close(p1[1]);
    close(p2[0]);

    // Affichage synthétique des résultats
    printf("=====\n");
    printf("[Bilan Père] Validation finale des calculs :\n");
    for (int i = 0; i < 5; i++) {
        printf(" - Entrée: %2d ==> Sortie Fils: %3d (Validation: %s)\n",
            valeurs_test[i], resultats[i],
            (valeurs_test[i] * 2 == resultats[i]) ? "OK" : "ÉCHEC");
    }
    printf("=====\n");
}

return EXIT_SUCCESS;
}
```

## 5. Conclusion Générale

---

Ce travail pratique a permis d'acquérir une expérience concrète et approfondie des couches bas niveau de communication et de gestion asynchrone en environnement Unix. Nous avons d'une part maîtrisé la capture des signaux POSIX (`sigaction`) en garantissant la sécurité d'exécution par l'usage exclusif de primitives réentrantes, évitant ainsi les blocages critiques inhérents aux interruptions de signaux.

D'autre part, la manipulation des tubes nous a sensibilisés aux contraintes physiques des flux de données partagés par le noyau. Nous avons analysé et observé la terminaison par `SIGPIPE` lors de l'écriture sans lecteurs, et conçu avec succès un canal d'échange bidirectionnel stable, structuré et exempt de blocage mutuel (deadlock).

Ces notions fondamentales d'IPC (Inter-Process Communication) et de programmation événementielle constituent la base technique nécessaire pour appréhender les concepts avancés de réseau (Sockets) et de développement distribué multi-processus/multi-threads.